Galgotias College
of
Engineering
and
Technology

# Assignment

Name : Nitish Kr. Yadav

Subject : Operating System

Year/Sem : I$^{st}$/ II$^{nd}$

Roll no : 2500970140124

Submitted to : Mr. Neeraj Kaushik

# Assignment 1

**Q1** Compare and contrast the layered Operating System structure with the monolithic structure. Which structure is more suitable for a security-critical application? Justify your answer.

◇ An operating system can designed using different structure Two common approaches are the layered structure and the monolithic structure.

* Layered structure

In the layered structure, the OS is divided into multiple layers arranged in a hierarchical manner. Each layer perform specific function and interact with only its adjacent layers.

. The lowest layer interact with hardware

. The highest layer interact with user.

advantages:

- Modular design makes system easy to understant
- easier debugging and maintenance
- better security due to isolation between layers.
- error in one layer do not easily affect others.

disadvantages:

- performance may be slower due to layer-by layer communication.
- designing proper layer can be difficult

2 Monolithic structure:

In this structure, the entire OS works as a single large program where all components are integrated and can directly interact with each other.

advantages:

- high performance due to direct communication
- faster execution
- simple design.

disadvantages:

- difficult to debug and maintain.
- lack of modularity
- a failure in one component can affect the whole system
- lower ~~strict~~ security due to strict boundaries.

for security critical applications, the layered operating system structure is more suitable

justification:

- it provide isolation so a ~~bug~~ breach in one layer does not affect the entire system.
- ensure controlled information between layers.
- make it easier to detect and fix vulnerabilities
- support better testing and verification of individual components.

Q2. Discuss how the requirement of a Real time operating system (RTOS) differ from those of Time sharing operating system. provide a suitable application scenarios. to support your comparison.

1. Real time operating system:

An RTOS is designed to process data and provide output within a fixed period of time constraint (deadline). The correctness of the system depends not only on logical results but also on the time at which result are produced.

Key requirements:

- Strict timing constraints (deadlines must be met)
- predictable and deterministic behaviour.
- fast response to external events.
- High reliability and stability
- priority based scheduling.

Types:

- Hard real time: missing deadline = system failure
- Soft real time: Missing deadline reduces performance but not failure.

2 Time sharing System:

A time sharing os allows multiple user or processes to share CPU time by dividing it into small time slices (quantum). each user get a chance to execute, creating an illusion of parallelism.

Key requirement:

- fair CPU allocation among users.
- good response time (not strict deadlines)
- efficient resource utilization.
- support for multitasking.

Application Scenarios:

RTOS Applications:

- Air traffic control system
- medical devices (e.g. heart monitors)
- industrial automation system.
- Automotive system (airbags, ABS).

in these cases, missing a deadline can cause serious damage or loss of life, so strict timing is essential.

Time sharing System applications:

- multiuser computer system.
- Online servers and databases
- desktop operating system.
- Educational lab systems

here the goal is to serve multiple users efficiently, not to meet strict deadlines.

RTOS and time sharing are mainly differs in their approach to time management. RTOS emphasizes strict timing and reliability while Time sharing system focuses on fairness and efficient resource utilization for general-purpose computing

3. Describe the concept of a Reentrant kernel. How does reentrancy enhance the performance of and reliability of multi-threaded applications?

A Reentrant kernel is a type of operating system kernel that allows multiple processes or threads to execute kernel code simultaneously without causing interference or data corruption.

A function or code is said to be reentrant if:

- it can be safely executed by multiple threads at the same time.
- it does not use shared data in an unsafe way
- it does not modify global variables (or uses proper synchronization)

How Reentrancy Enhances performance:

1) parallel execution.
- multiple threads can run kernel code simultaneously
- improves CPU utilization.

2) Better throughout
more tasks are completed in less time

3) Reducing waiting time
Threads do not have to wait for others to finish kernel execution.

4) Efficient multi-core usage:
takes full advantage of modern multi-core processors.

How Reentrancy Improves Reliability:

1. Avoids data Corruption
   Safe handling of shared resources prevents inconsistencies
2. Thread Safety
   multiple threads can operate without interfering with each other.
3. Stable System behaviour
   Reduces chances of crashes and unexpected errors.
4. Better fault Isolation
   errors in one thread are less likely to affect others

Q4 Discuss how OS services such as memory management, file system and I/O control are interdependent. Use a case scenario of opening and editing a file stored on a remote server.

Interdependence of OS Services:

Operating system services like memory management file system, and I/O control are closely interconnected and work together to perform task efficiently

1) file system:
   - organizes and manages files on storage devices
   - Handles operation like open, read, write, save
   - maintain file location and permissions.

2. Memory management:
- Allocates RAM to process and programs.
- loads file data into memory for processing

3 I/O control
→ manages communication between hardware devices and OS.
- Handles input/output operation. (disk, network, keyword.)

Case Scenario: Opening & Editing a file from remote server:

1 File request
opens a file stored on a remote server

2 Data transfer
Data is transferred from the remote server to local system.

3. loading into memory
memory is allocated to the application

4. Editing process
changes are made in memory

5. Saving file
file system update the files on the server.

Q.6 What is the critical section problem and why it is important in concurrent programming? Explain how dekkers or patterson solution can be used to let the processes safely share an resource. Write a simple program to show how it works.

Ans Critical section problem.

It occurs when multiple process try to access shared resource at the same.

Why important?

* prevent data inconsistency
* Avoid race condition.

peterson's solution (for a process)

```
boolean flag[2];
int turn;

process 0;
flag[0] = turn;
turn = 1
while (flag[1] && turn == 1);
// Critical Section
flag[0] = false;
```

⇒ Ensures only one process enters critical section at a time.

Q.6 What is the critical section problem and why it is important in concurrent programming? Explain how dekkers or petersons solution can be used to let the processes safely share an resource. Write a simple program to show how it works.

Ans Critical section problem.

It occurs when multiple process try to access shared resource at the same.

Why important?

* prevent data inconsistency
* Avoid race condition.

petersons solution (for 2 process)

```
boolean flag[2];
int turn;

process 0;
flag[0] = turn;
turn = 1
while (flag[1] && turn == 1);
// Critical Section
flag[0] = false;
```

⇒ Ensures only one process enters critical section at a time.

Q6 What are semaphores and test and Set operation and how do they help in controlling access to shared resources? use a simple producer-consumer problem example to show how are of them works in real life.

Semaphore:
A variable used to control access to shared resources

Types:
* Binary Semaphore (0 or 1)
* Counting Semaphore

Test and Set:
Hardware instruction for locking:

producer - consumer using Semaphore:

```
Semaphore full = 0;
Semphor empty = 1;
Semaphore mutex = 1;

// producer
  wait (empty);
  wait (mutex);
```

// producer item
Signal (mutex);
Signal (full);

// consumer
wait (full);
wait (mutex);

// consume item
Signal (mutex);
Signal (empty);

prevents overflow and underflow.

show the difficulties of using shared resources. choose one of these problems explain it in simple words, and give a solution that avoids deadlock and starvation?

The Dining Philosopher problem

Imagine five philosopher sitting around a circular table between each pair of adjacent philosopher lies a single chopstick - so there are five chopstick total. each philosopher alternates between two activities: thinking and eating. To eat, a philosopher must pick up both the chopstick on the left and the chopstick on the right.

The trouble begins with every philosopher simultaneously picks up their chopstick. now each one is holding one chopstick and waiting for the right one - which is already held by their neighbour. nobody can eat. Nobody will let go. This is deadlock.

Starvation is a related but different problem; even without deadlock, a philosopher might be repeatedly skipped over and let get to eat, while others. feast.

Conditions for deadlock:

Deadlock requires all four of these conditions to hold simultaneously (known as the coffman conditions)

mutual exclusion: a chopstick can only be held by one philosopher at a time

Hold and wait: a philosopher holds one chopstick while waiting for another

No preemption: a chopstick can't be forcibly taken away.

Circular wait: $P_0$ waits for $P_1$, $P_1$ waits for $P_2$ .... $P_4$ for $P_0$. broken by forcing philosopher 4 to pick up chopstick 0 before chopstick 4, the circle is cut. There is always at least one philosopher whose "first chopstick" nobody else is waiting on, so the chain can never close into loop

Ques 8. What is Inter-process Communication (IPC) and how do processes talk to each other in an operating system? Create a small idea or example for a chat application where different users (processes) can send and receive messages using IPC.

Every program running on your country is a process - an isolated bubble of memory managed by the OS. processes cannot directly read others memory by designs this isolation is a core safety feature but real application need to cooperate - a web server needs to hands work to a database, a music app need to talks to a system audio daemon user on a chat app needs to exchange messages. IPC is the set of mechanism the OS provides to let process talk to each other safely